

RELATÓRIO DE DESENVOLVIMENTO E ANÁLISE CRÍTICA

Detecção de Danos em Carros com Deep Learning:

uma comparação entre YOLOv8, Faster R-CNN e Mask R-CNN

Cauê Eberspacher Menezes

Visão Computacional — UNIFESP

Projeto SAR — Junho de 2026

Sumário

Sumário	2
1. Introdução e Objetivo do Projeto.....	4
2. Configuração do Ambiente de Desenvolvimento	4
Análise crítica	5
Backpropagation/Retropropagação e Treino de Redes Neurais	5
Redes Neurais Convolucionais	5
3. A Base de Dados: COCO Car Damage	6
Análise crítica	8
4. Implementação e Treinamento dos Três Modelos	8
4.1 YOLOv8 (yolotraining.py).....	8
4.2 Faster R-CNN (faster_r_cnntraining.py)	8
4.3 Mask R-CNN (mask_r_cnntraining.py)	9
Resumo da primeira rodada completa de resultados.....	9
5. Primeira Apresentação e Falhas Identificadas	9
6. Correções Estruturais do Projeto	10
6.1 Barra de progresso e avaliação completa	10
6.2 Script de comparação visual (comparacao.py)	10
6.3 Menu central (main.py)	10
6.4 Caminhos relativos e portabilidade.....	11
7. Materiais de Apoio à Defesa	11
7.1 Apresentação em slides.....	11
7.2 Roteiro de defesa oral	11
7.3 Diagrama de casos de uso	12
8. Fundamentação Teórica Adicional	12
8.1 Hiperparâmetros	12
8.2 Gradiente Descendente Estocástico (SGD)	12
8.3 Deformação do espaço de características (space warping)	13
8.4 K-Means como possível complemento	13
9. Problemas de Infraestrutura e Organização de Arquivos	13
Análise crítica	14
10. Planilha de Métricas e Cálculo de Precision, Recall e F1.....	14
11. Ambiente Visual Studio e Compatibilidade de Interpretador	15
12. O Bug Crítico na Avaliação do Mask R-CNN	15
Análise crítica	16

13. Resultados Finais Consolidados.....	16
Interpretação dos resultados finais por métrica.....	16
14. Controle de Versão com GitHub.....	17
15. Análise Crítica Final e Trabalhos Futuros	17
Sobre reprodutibilidade	17
Sobre comunicação técnica.....	17
Sobre as limitações conhecidas do trabalho	17
Trabalhos futuros	18

1. Introdução e Objetivo do Projeto

O projeto SAR (carros amassados) tinha como proposta original buscar uma ou mais bases de dados de carros danificados na internet, implementar três métodos de deep learning capazes de localizar os danos nas imagens, treinar cada rede e, por fim, comparar quantas imagens cada uma acertou e errou, separando um conjunto de imagens fora do treinamento para o teste final.

Este relatório documenta, em ordem cronológica e com análise crítica, todo o processo de implementação, correção e validação que percorri para cumprir essas etapas — desde a configuração inicial do ambiente de desenvolvimento até a consolidação dos resultados finais, passando por uma apresentação que não foi aprovada na primeira tentativa e pelas correções estruturais que vieram depois dela.

Mais do que um registro técnico de comandos executados, este documento busca registrar também as decisões tomadas, os erros cometidos e o raciocínio por trás de cada correção — elemento que considero central para demonstrar domínio crítico sobre o próprio trabalho, e não apenas a capacidade de fazer um programa funcionar.

2. Configuração do Ambiente de Desenvolvimento

Antes de escrever qualquer linha de código relacionada diretamente ao projeto, foi necessário resolver uma série de problemas de infraestrutura que, embora não pareçam centrais ao tema de deep learning, consumiram boa parte do tempo inicial e revelaram a importância de um ambiente de desenvolvimento corretamente configurado.

- O comando pip não era reconhecido pelo PowerShell: na verdade, o Python não estava de fato instalado no sistema, apenas um atalho do Windows para a Microsoft Store.
- A tentativa de instalação pela Microsoft Store (Python Install Manager) falhou com o erro 0x80070020, relacionado a um conflito de registro de pacote.
- A solução definitiva foi a instalação manual a partir do instalador oficial disponível em python.org (versão 3.14.5), marcando explicitamente a opção “Add python.exe to PATH” durante a instalação.
- No PyCharm, configurei o interpretador do projeto como um ambiente virtual (Virtualenv) próprio, com a opção “Inherit packages from base interpreter” marcada, garantindo que esse ambiente herdasse as bibliotecas já instaladas no Python base.
- Instalei as bibliotecas centrais do projeto em etapas: inicialmente pandas, numpy, matplotlib, seaborn, scikit-learn e openpyxl; posteriormente, já com o foco em deep learning, ultralytics, torch, torchvision, opencv-python e tqdm.

Análise crítica

Esse processo, ainda que trabalhoso, deixou clara uma lição que se repetiu várias vezes ao longo do projeto: erros de ambiente (variável PATH, interpretador incorreto, bibliotecas ausentes) são frequentemente confundidos com erros de lógica do programa. A primeira pergunta diante de qualquer falha de execução deveria ser “o ambiente está configurado corretamente?” antes de revisar o código em si — uma lição que voltaria a se confirmar, mais adiante, ao tentar rodar o projeto a partir de uma segunda IDE (Visual Studio).

Backpropagation/Retropropagação e Treinamento de Redes Neurais

A capacidade de aprendizado dos modelos utilizados neste trabalho (YOLOv8, Faster R-CNN e Mask R-CNN) baseia-se fundamentalmente no algoritmo de **retropropagação** (*backpropagation*). Como explicado por Justin Johnson (Lecture 6, Michigan Online, 2019), a retropropagação aplica de forma eficiente a regra da cadeia (*chain rule*) para calcular o gradiente da função de perda em relação a cada peso da rede. Esse processo permite propagar o erro da camada de saída até as camadas iniciais, possibilitando a atualização dos milhões de parâmetros por meio de otimizadores como o Gradiente Descendente Estocástico (SGD).

No contexto deste projeto, a retropropagação é o mecanismo central que permite transformar pixels brutos das imagens SAR em representações hierárquicas úteis. As camadas iniciais da rede aprendem detectores de bordas e texturas, enquanto as camadas mais profundas combinam essas características em padrões semanticamente relevantes (“dano” versus “sem dano”). Esse processo envolve conceitos como *space warping* (deformação do espaço de features por meio de transformações lineares seguidas de não-linearidades, como a função ReLU) e a otimização iterativa dos pesos, controlada por hiperparâmetros como *learning rate*, *momentum* e *weight decay*.

Além disso, a estrutura convolucional das CNNs resolve uma limitação crítica dos classificadores lineares simples: ao contrário do alongamento da imagem em um vetor único (*flatten*), que descarta a estrutura espacial, as operações de convolução preservam e exploram a disposição bidimensional dos pixels, tornando viável a detecção eficiente de amassados.

Redes Neurais Convolucionais (CNN's)

Um dos principais avanços na Visão Computacional foi o desenvolvimento das Redes Neurais Convolucionais (CNNs), que superam as limitações dos classificadores lineares fully-connected. Como destacado por Justin Johnson (Lecture 7, Michigan Online, 2019), ao contrário do procedimento de *flatten* (alongamento da imagem em um vetor unidimensional), que descarta a estrutura espacial dos dados, as CNNs operam diretamente sobre a disposição bidimensional dos pixels por meio de operações locais.

Os componentes essenciais de uma CNN incluem:

- **Camadas de Convolução:** Aplicam filtros (kernels) que deslizam sobre a imagem, computando produtos escalares em regiões locais. Cada filtro aprende a detectar padrões específicos (bordas, texturas, orientações). Filtros sucessivos geram mapas de ativação,

aumentando progressivamente o *receptive field* (campo receptivo) — a porção da imagem original que influencia um neurônio em camadas mais profundas.

- **Funções de Ativação** (ex.: ReLU): Introduzem não-linearidade, permitindo que a rede aprenda funções complexas (*space warping*).
- **Camadas de Pooling** (tipicamente Max Pooling): Reduzem a resolução espacial, conferem invariância a pequenas translações e diminuem o custo computacional, sem adicionar parâmetros aprendíveis.
- **Batch Normalization**: Normaliza as ativações de cada mini-batch (média zero e variância unitária), acelerando o treinamento, permitindo taxas de aprendizado maiores, atuando como regularizador e mitigando o *internal covariate shift*.

A arquitetura clássica segue o padrão [Conv + ReLU + Pool] × N, seguido de camadas fully-connected para classificação. Exemplos históricos como a LeNet-5 (LeCun et al., 1998) já demonstravam essa estrutura, que foi refinada em arquiteturas modernas como ResNet e os backbones utilizados em YOLOv8, Faster R-CNN e Mask R-CNN.

No presente projeto, esses princípios são diretamente aplicados: os backbones convolucionais extraem características hierárquicas das imagens SAR, transformando pixels brutos em representações onde regiões com amassados tornam-se distinguíveis. A retropropagação (*backpropagation*) propaga o erro através dessas camadas, otimizando os pesos via Gradiente Descendente Estocástico, enquanto o Batch Normalization contribui para a estabilidade do treino mesmo com datasets limitados.

3. A Base de Dados: COCO Car Damage

A base de dados principal utilizada no projeto foi a COCO Car Damage, estruturada no formato de anotação COCO (bounding box e segmentação poligonal), com uma única categoria de objeto: “damage” (dano).

Característica	Valor
Imagens de treino	59
Imagens de validação	11
Total de imagens	70
Anotações de treino	140
Anotações de validação	24
Total de anotações	164
Resolução das imagens	1024 × 1024 px
Média de danos por imagem	≈ 2,3

Também identifiquei e baixei uma segunda base de dados, a CarDD, com aproximadamente 4.000 imagens anotadas no mesmo padrão COCO. Por seu tamanho (mais de 5 GB) e pelas limitações de hardware disponíveis para o treinamento (apenas CPU, sem GPU dedicada), optei por não utilizá-la no treinamento principal, registrando-a como possibilidade de trabalho futuro caso uma GPU esteja disponível.

A primeira função implementada, no script `datareading.py`, teve como objetivo carregar os arquivos de anotação (`COCO_train_annos.json` e `COCO_val_annos.json`), exibir estatísticas básicas do dataset no console e sobrepor visualmente, sobre as imagens, os retângulos (bounding boxes) correspondentes aos danos anotados, utilizando a biblioteca `matplotlib.patches`. Essa etapa, apesar de simples, foi essencial para confirmar que as anotações estavam corretamente alinhadas com as imagens antes de qualquer treinamento.

Análise crítica

A primeira versão do `datareading.py` exibía apenas uma única imagem (a primeira do conjunto de validação) com seus danos marcados. Esse foi, mais tarde, um dos pontos apontados como insuficiente na apresentação: a exploração do dataset deveria contemplar o conjunto completo de imagens, não apenas uma amostra isolada. A versão corrigida do script passou a salvar todas as 11 imagens de validação anotadas na pasta `resultados_dataset/`, permitindo uma inspeção visual completa do conjunto de teste antes mesmo de qualquer treinamento.

4. Implementação e Treinamento dos Três Modelos

Conforme a proposta original do trabalho, implementei três métodos distintos de deep learning para a tarefa de detecção de danos, cada um com uma arquitetura e abordagem diferentes, de forma a permitir uma comparação significativa entre eles.

4.1 YOLOv8 (`yolotraining.py`)

O YOLOv8 é um modelo de detecção de objetos em uma única etapa: ele processa a imagem inteira de uma só vez para prever simultaneamente a localização e a classe dos objetos, o que o torna o mais rápido dos três métodos. Como o dataset estava no formato COCO e o YOLOv8 (via biblioteca Ultralytics) exige um formato próprio (arquivos `.txt` com coordenadas normalizadas), a primeira parte do script foi dedicada a uma função de conversão (`converter_coco_para_yolo`), que lê cada anotação COCO e grava o arquivo de rótulo YOLO correspondente, no padrão classe `x_centro y_centro largura altura`, com valores normalizados entre 0 e 1.

O treinamento utilizou o modelo pré-treinado YOLOv8n (variante “nano”, com cerca de 3 milhões de parâmetros) por até 50 épocas, com parada antecipada (`EarlyStopping`, `patience=10`) caso não houvesse melhora. Na primeira rodada completa, o treinamento parou na época 13 (melhor resultado obtido na época 3), e a avaliação nas imagens de validação não identificou nenhum dos 24 danos reais — um resultado de 0% de acerto.

Esse resultado, embora desapontador a princípio, tornou-se um dado relevante para a análise comparativa: evidenciou que um dataset de apenas 59 imagens de treino é insuficiente para que um modelo de uma única etapa, como o YOLOv8, generalize bem de forma consistente, mesmo utilizando pesos pré-treinados como ponto de partida.

4.2 Faster R-CNN (`faster_r_cntraining.py`)

O Faster R-CNN é um modelo de duas etapas: primeiro, uma sub-rede chamada Region Proposal Network (RPN) sugere regiões candidatas a conter objetos; em seguida, uma segunda etapa classifica cada região e refina a posição do retângulo previsto. Para esse modelo, implementei uma classe própria de dataset (`CarDamageDataset`, baseada em `torch.utils.data.Dataset`) responsável por carregar as imagens e converter as anotações COCO diretamente em tensores de bounding boxes e rótulos, sem necessidade de conversão de formato como no caso do YOLOv8.

Utilizei a arquitetura `fasterrcnn_resnet50_fpn` da biblioteca `torchvision`, com pesos pré-treinados no dataset COCO, substituindo apenas a camada final de classificação (`FastRCNNPredictor`) para reconhecer duas classes: fundo e dano. O treinamento foi conduzido por 15 épocas utilizando o otimizador SGD (Stochastic Gradient Descent), com taxa de aprendizado de 0,005, momentum de 0,9 e weight decay de 0,0005. Na primeira rodada completa, o modelo atingiu 62,5% de acerto (15 de 24 danos reais identificados na validação).

4.3 Mask R-CNN (`mask_r_cnntraining.py`)

O Mask R-CNN estende o Faster R-CNN adicionando uma terceira saída à rede: além da caixa delimitadora e da classe, o modelo também prevê uma máscara de segmentação — um mapa de pixels que indica o contorno exato do objeto, e não apenas um retângulo aproximado. Isso exigiu adaptar a classe de dataset para também converter as anotações de segmentação poligonal do COCO em máscaras binárias (utilizando `cv2.fillPoly`), além de utilizar a arquitetura `maskrcnn_resnet50_fpn` e substituir tanto o classificador de caixas quanto o preditor de máscaras (`MaskRCNNPredictor`).

Treinado também por 15 épocas, com os mesmos hiperparâmetros do Faster R-CNN, o Mask R-CNN obteve, na primeira rodada completa, o melhor resultado entre os três métodos: 83,3% de acerto (20 de 24 danos reais). A hipótese para esse desempenho superior é a de que, ao ser forçado a aprender também o contorno exato do dano — e não apenas uma caixa retangular —, o modelo desenvolve uma representação interna mais rica da forma dos amassados, refletida em uma detecção mais precisa mesmo quando avaliada apenas pela presença ou ausência de caixas.

Resumo da primeira rodada completa de resultados

Modelo	Acertos/Total	Taxa de acerto	Loss final	Épocas	Tempo de execução
YOLOv8	0/24	0%	—	13 (early stop)	4m41s
Faster R-CNN	15/24	62,5%	0,1066	15	51m22s
Mask R-CNN	20/24	83,3%	0,2492	15	49m11s

5. Primeira Apresentação e Falhas Identificadas

A primeira apresentação do trabalho não foi aprovada pela banca. Ao refletir sobre o que faltou, identifiquei três falhas estruturais centrais, além do conteúdo técnico em si:

- Falta de material visual de apoio: a apresentação não contava com slides ou diagramas que explicassem visualmente as etapas e funções do programa, o que dificultou a compreensão da banca sobre o funcionamento do projeto.
- Funções incompletas: o treinamento por épocas era exibido apenas parcialmente no console, sem uma barra de progresso clara, e a função de visualização de detecções

mostrava o resultado em apenas uma única imagem, quando o ideal seria demonstrar o comportamento do modelo sobre o conjunto completo de imagens de teste.

- Um dos scripts não executava de forma autônoma fora do ambiente em que foi originalmente desenvolvido, pois dependia do download de pesos pré-treinados via internet sem qualquer tratamento de erro ou aviso ao usuário.

Essas três falhas, embora pareçam superficiais frente ao conteúdo de deep learning em si, refletem um problema mais profundo: um projeto técnico só é avaliado de forma justa quando pode ser demonstrado e compreendido pela banca, e não apenas quando “funciona” do ponto de vista do autor. A seção seguinte documenta as correções implementadas para sanar exatamente esses três pontos.

6. Correções Estruturais do Projeto

6.1 Barra de progresso e avaliação completa

Para resolver o problema do treinamento exibido parcialmente, adicionei a biblioteca `tqdm` aos três scripts de treinamento, substituindo o laço de treinamento simples por um laço decorado com barra de progresso (`tqdm(train_loader, desc=f"Época {epoch+1}/{EPOCHS}")`), que exibe em tempo real o percentual de conclusão de cada época, o tempo estimado restante e a perda (loss) do lote atual.

Para resolver o problema da detecção restrita a uma única imagem, reescrevi a etapa de avaliação dos três scripts de treinamento para percorrer todas as imagens do conjunto de validação, gerando, para cada uma, uma figura comparativa com dois painéis — o gabarito real (ground truth, em verde) e a predição do modelo (em vermelho, azul ou roxo, dependendo do modelo) — e salvando automaticamente o resultado em uma pasta de resultados própria para cada modelo (`resultados_yolo/`, `resultados_fasterrcnn/`, `resultados_maskrcnn/`).

6.2 Script de comparação visual (`comparacao.py`)

Criei um quarto script, independente dos três scripts de treinamento, cuja única responsabilidade é carregar os três modelos já treinados (a partir dos arquivos `.pt` salvos) e executar a predição de todos eles sobre a mesma imagem, dispondo o resultado em uma única figura com quatro painéis lado a lado: o gabarito real e a predição de cada um dos três modelos. Esse script depende, portanto, da existência prévia dos arquivos `fasterrcnn_best.pt`, `maskrcnn_best.pt` e do checkpoint `best.pt` gerado pelo YOLOv8 dentro da pasta `runs/detect/`.

6.3 Menu central (`main.py`)

Para facilitar a execução do projeto como um todo — e evitar que o avaliador precisasse abrir e executar manualmente cada um dos cinco scripts —, implementei um script `main.py` que apresenta um menu numerado em texto no console, permitindo escolher entre explorar o dataset,

treinar cada um dos três modelos individualmente ou executar a comparação visual, tudo a partir de um único ponto de entrada. Internamente, o menu utiliza `subprocess.run()` para invocar cada script como um processo Python separado, herdando o mesmo interpretador (`sys.executable`) e o mesmo diretório de trabalho.

6.4 Caminhos relativos e portabilidade

O problema do script que “não rodava offline” tinha, na verdade, duas causas distintas, ambas relacionadas à portabilidade do código entre máquinas diferentes. A primeira foi resolvida simplesmente executando o script de treinamento uma vez com conexão à internet, o que faz com que a biblioteca `torchvision` armazene em cache local os pesos pré-treinados, tornando as execuções seguintes independentes de internet.

A segunda causa, mais estrutural, foi a substituição de todos os caminhos absolutos fixos (como `r"C:\CODIGOSAR\..."`, escritos diretamente no código) por caminhos relativos calculados dinamicamente a partir da localização do próprio script, segundo o padrão:

```
DIR = os.path.dirname(os.path.abspath(__file__)) BASE = os.path.join(DIR, "..", "COCO Car  
Damage (Projeto SAR)")
```

Essa mudança garante que o projeto funcione corretamente em qualquer computador, independentemente do nome do usuário ou da letra da unidade de disco, desde que a estrutura relativa de pastas seja mantida.

7. Materiais de Apoio à Defesa

7.1 Apresentação em slides

Produzi uma apresentação com sete telas, cobrindo: (1) capa; (2) objetivo do projeto e aplicações práticas, como seguradoras, locadoras e oficinas; (3) caracterização da base de dados; (4) explicação comparativa das arquiteturas dos três modelos; (5) fundamentos teóricos do treinamento (hiperparâmetros e deformação do espaço de características); (6) resultados comparativos em gráfico de barras e cartões de destaque; e (7) conclusão. Cada slide inclui anotações de apresentador com lembretes do que destacar verbalmente durante a defesa.

7.2 Roteiro de defesa oral

No início tive essa ideia de usar um roteiro de defesa oral em 15 minutos, como numa banca de graduação, mas abandonei e preferi optar por este relatório, por que acho que seria melhor do que ler uma coisa literalmente e ficar com uma dúvida que eu não saiba responder ao professor ou aos colegas que não está escrito no roteiro. Pessoalmente, creio que fiz o correto.

7.3 Diagrama de casos de uso

Para complementar a documentação técnica, elaborei um diagrama UML de casos de uso, identificando dois atores (Pesquisador e Avaliador) e cinco casos de uso principais (Carregar dataset, Visualizar anotações, Treinar modelo, Detectar danos e Comparar modelos), com uma relação de inclusão («include») entre “Detectar danos” e “Treinar modelo”, já que a detecção depende logicamente da existência de um modelo previamente treinado.

8. Fundamentação Teórica Adicional

Buscando aprofundar a qualidade da explicação técnica do trabalho — outro dos pontos identificados como insuficiente na primeira apresentação —, organizei quatro conceitos centrais de deep learning presentes na implementação, todos vinculados a trechos concretos do código.

8.1 Hiperparâmetros

Distintos dos pesos da rede (que são ajustados automaticamente durante o treinamento), os hiperparâmetros são valores definidos manualmente antes do treino e controlam como o aprendizado ocorre.

Hiperparâmetro	Valor utilizado	Função
Taxa de aprendizado (lr)	0,005	Tamanho do passo de cada ajuste de peso
Momentum	0,9	Acelera a convergência, suaviza oscilações
Weight decay	0,0005	Penaliza pesos muito grandes (evita overfitting)
Batch size	2 a 4	Quantidade de imagens processadas por vez
Épocas	15 (50 c/ early stop no YOLOv8)	Número de passagens completas pelo dataset
Limiar de confiança (conf)	0,5 (R-CNNs) / 0,1 (YOLOv8)	Confiança mínima para validar uma detecção

8.2 Gradiente Descendente Estocástico (SGD)

O algoritmo responsável por ajustar os pesos da rede durante o treinamento do Faster R-CNN e do Mask R-CNN foi o SGD (torch.optim.SGD). A cada lote de imagens, o algoritmo calcula o gradiente da função de perda (loss) em relação a cada peso da rede — ou seja, a direção que mais aumentaria o erro — e atualiza os pesos no sentido contrário, reduzindo o erro. É chamado “estocástico” porque utiliza apenas uma pequena amostra dos dados (o lote) a cada atualização, em vez do conjunto de treino inteiro, tornando o processo mais rápido e computacionalmente viável.

8.3 Deformação do espaço de características (space warping)

Um modo de visualizar o que cada camada de uma rede neural faz, internamente, é pensar nela como uma transformação geométrica do espaço dos dados. Cada camada aplica uma transformação linear seguida da função de ativação ReLU:

$$h = \text{ReLU}(Wx) = \max(0, Wx)$$

Em um exemplo simplificado bidimensional, pontos de duas classes inicialmente organizados de forma não separável por uma linha reta (por exemplo, uma classe formando um anel em torno da outra) podem, após sucessivas transformações desse tipo, reorganizar-se em uma configuração onde uma única linha reta os separa. É esse princípio — repetido em muitas camadas — que permite às redes convolucionais ResNet50, utilizadas como base do Faster R-CNN e do Mask R-CNN, transformarem pixels de uma imagem em representações onde a presença ou ausência de dano se torna uma distinção simples para a camada classificadora final.

8.4 K-Means como possível complemento

Durante o desenvolvimento, considerei a possibilidade de aplicar o algoritmo de agrupamento K-Means como uma análise complementar — por exemplo, agrupando as anotações de dano por tamanho e formato do retângulo delimitador, para identificar automaticamente categorias naturais de danos (pequenos, médios e grandes) no dataset, ou agrupando regiões de pixels recortadas por similaridade de cor, como forma de distinguir, por exemplo, arranhões superficiais de amassados mais profundos. Por se tratar de uma técnica de aprendizado não supervisionado, diferente dos três métodos supervisionados já implementados, optei por não incorporá-la ao núcleo do trabalho nesta etapa, registrando-a como uma extensão natural para trabalhos futuros.

9. Problemas de Infraestrutura e Organização de Arquivos

Ao longo das correções pós-apresentação, enfrentei uma série de problemas relacionados à organização dos arquivos do projeto, todos derivados de uma mesma causa raiz: mover arquivos manualmente pelo Explorer do Windows, em vez de utilizar o painel de projeto do próprio PyCharm, que atualiza automaticamente as referências internas.

- Os scripts `comparacao.py` e `main.py` foram, em determinado momento, criados ou movidos para dentro da pasta `runs/` (e, em um dos casos, `runs/runs/`), criada automaticamente pelo YOLOv8 para armazenar os pesos de treinamento. Como os caminhos relativos pressupõem uma posição fixa do script (uma pasta acima da raiz do projeto), essa movimentação indevida gerou repetidos erros do tipo `FileNotFoundError`.
- O arquivo `fasterrcnn_best.pt` (modelo treinado) também foi encontrado, em certo momento, dentro de `Código/runs/`, e não na raiz do projeto onde era esperado — exigindo

localizá-lo manualmente com a ferramenta de busca do Windows e movê-lo de volta ao lugar correto.

- O dicionário de etapas do `main.py` referenciava os nomes `fasterrcnn_train.py` e `maskrcnn_train.py`, enquanto os arquivos reais do projeto sempre haviam sido nomeados `faster_r_cnntraining.py` e `mask_r_cnntraining.py` — uma divergência de nomenclatura que gerou erros de “arquivo não encontrado” até ser identificada e corrigida diretamente no dicionário do `main.py`, mantendo os nomes originais dos arquivos para evitar uma nova rodada de ajustes em cascata.
- Um erro de sintaxe no próprio `main.py` — um “12” residual ao final de uma linha de código, provavelmente proveniente de uma cópia acidental do número de linha exibido por um visualizador de código — impediu a execução do script até ser identificado e removido.

Análise crítica

Esses problemas, embora triviais a posteriori, evidenciam a importância de se utilizar as ferramentas de refatoração de uma IDE (como mover e renomear arquivos pelo próprio PyCharm) em vez de operações diretas no sistema de arquivos, já que apenas a IDE consegue atualizar automaticamente todas as referências internas do projeto a um arquivo movido ou renomeado.

10. Planilha de Métricas e Cálculo de Precision, Recall e F1

Paralelamente ao desenvolvimento do código, organizei uma planilha em Excel para consolidar as estatísticas do dataset, os resultados de cada modelo, o tempo de treinamento e os resultados de teste final em imagens separadas do treinamento. As métricas de avaliação, calculadas a partir dos números absolutos de detecções e acertos retornados por cada script, foram:

- $\text{Precision} = \text{Acertos} \div \text{Detecções totais}$ (entre as detecções feitas pelo modelo, quantas estavam corretas)
- $\text{Recall} = \text{Acertos} \div \text{Danos reais totais}$ (entre os danos que realmente existem, quantos o modelo conseguiu encontrar) — corresponde à “taxa de acerto” reportada pelos scripts
- $\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) \div (\text{Precision} + \text{Recall})$ — uma média harmônica que equilibra as duas métricas anteriores

Optei por não calcular a métrica de IoU (Intersection over Union, que mede a sobreposição geométrica entre a caixa prevista e a caixa real) por ela exigir uma comparação par a par entre coordenadas de caixas, algo que os scripts de avaliação implementados não realizam — eles apenas contam quantas detecções, no total, correspondem a danos reais, sem associar cada detecção a uma anotação específica. Considero essa uma limitação conhecida e documentada do método de avaliação utilizado, e não um erro: para os fins comparativos do trabalho, a contagem de detecções por confiança mínima foi suficiente para revelar diferenças relevantes de comportamento entre os três modelos.

11. Ambiente Visual Studio e Compatibilidade de Interpretador

Ao tentar executar os scripts também a partir do Visual Studio 2026 (software gratuito) — para aproveitar a maior área de visualização do terminal integrado durante a gravação dos vídeos de benchmark —, todos os scripts falharam com o erro `ModuleNotFoundError`, apontando para bibliotecas como `torch` e `matplotlib`. A causa não estava no código, mas no ambiente: o Visual Studio, por padrão, utilizava o interpretador Python global do sistema (Python 3.14, instalado diretamente no Windows), e não o ambiente virtual (`.venv`) criado anteriormente pelo PyCharm (versão trial) dentro da pasta do projeto, onde todas as bibliotecas necessárias haviam sido instaladas.

A solução foi registrar o ambiente virtual existente como um “Ambiente Existente” dentro do gerenciador de Ambientes Python do Visual Studio, apontando diretamente para o executável `C:\CODIGOSAR\.venv\Scripts\python.exe`, em vez de criar um novo ambiente virtual vazio. Essa configuração permitiu reutilizar todas as bibliotecas já instaladas, sem necessidade de reinstalação, e tornou possível rodar e gravar os treinamentos a partir de qualquer uma das duas IDEs de forma intercambiável.

12. O Bug Crítico na Avaliação do Mask R-CNN

O problema mais significativo identificado ao longo do processo de correção surgiu durante uma rodada de testes do Mask R-CNN, quando o resultado final reportado pelo script indicou “Danos reais (total): 6” — um valor inconsistente com todas as rodadas anteriores de todos os três modelos, nas quais esse número sempre havia sido 24 (o total real de anotações de dano nas 11 imagens de validação).

Diante da suspeita de um bug, e sem acesso direto ao código no momento (o terminal já havia sido encerrado, perdendo o histórico), recorri à análise de um trecho de vídeo gravado durante a execução. A inspeção quadro a quadro do vídeo revelou duas informações importantes: primeiro, que todas as 11 imagens de validação foram corretamente localizadas pelo script (confirmado por uma série de linhas de depuração “Existe?: True”), descartando a hipótese de um problema de caminho de arquivo; segundo, que essas mesmas linhas de depuração (“Caminho:”, “Arquivo:”, “Existe?:”) não faziam parte do código original fornecido — eram inserções manuais, provavelmente adicionadas durante uma sessão anterior de depuração de problemas de caminho, que aparentemente corromperam, de forma não intencional, a lógica de contagem de danos reais e detecções dentro do laço de avaliação.

A solução adotada foi descartar o arquivo editado manualmente e substituí-lo integralmente pela versão original e limpa do script, em vez de tentar localizar a linha exata responsável pelo erro dentro de um arquivo já modificado — um caminho de correção mais lento e propenso a deixar resíduos do problema. Na mesma oportunidade, adicionei aos três scripts de treinamento

(YOLOv8, Faster R-CNN e Mask R-CNN) uma funcionalidade de exportação automática do resumo final de resultados para um arquivo de texto (`resumo_final.txt`), eliminando a dependência de copiar manualmente o conteúdo do terminal antes que a janela fosse fechada — outro problema prático que se repetiu durante as sessões de benchmark.

Análise crítica

Esse episódio ilustra um risco real e recorrente em projetos sujeitos a modificações ad-hoc durante a depuração: alterações feitas “às pressas” para investigar um problema, como adicionar prints de depuração, podem elas mesmas introduzir um novo problema, silenciosamente, se não forem revertidas após o uso. A lição prática adotada foi a de, sempre que possível, manter uma cópia de referência limpa de cada script crítico, isolada de qualquer edição exploratória.

13. Resultados Finais Consolidados

Após a correção do bug e a repetição dos três treinamentos com os scripts corrigidos, os resultados finais, livres de inconsistências, foram:

Modelo	Deteções	Acertos	Precision	Recall	F1
YOLOv8	44	20/24	45,5%	83,3%	58,8%
Faster R-CNN	13	13/24	100%	54,2%	70,3%
Mask R-CNN	19	14/24	73,7%	58,3%	65,1%

Vale destacar que esses números diferem, em alguma medida, dos resultados da primeira rodada completa, na qual o Mask R-CNN havia atingido 83,3% de Recall, e não 58,3%. Essa diferença não decorre de qualquer alteração na arquitetura dos modelos ou no dataset, mas de dois fatores inerentes ao processo de treinamento de redes neurais: (1) a natureza estocástica do algoritmo SGD, que parte de pesos inicializados aleatoriamente e percorre os dados em uma ordem embaralhada a cada execução, podendo convergir para soluções diferentes em cada rodada; e (2) a mudança de hardware entre execuções (de um processador Intel Xeon E5-2680 para um Intel Core i7-13620H), que não afeta a acurácia em si, mas reforça a necessidade de cuidado ao comparar tempos de treinamento entre rodadas realizadas em máquinas diferentes.

Interpretação dos resultados finais por métrica

- Recall: o YOLOv8 obteve o melhor desempenho (83,3%), encontrando a maior parte dos danos reais, ao custo de um número expressivo de falsos positivos (44 detecções para apenas 24 danos reais existentes).
- Precision: o Faster R-CNN obteve o melhor desempenho, de forma absoluta (100%) — todas as 13 detecções que realizou estavam corretas, ainda que tenha deixado de identificar quase metade dos danos reais.
- F1 (equilíbrio entre as duas métricas anteriores): o Faster R-CNN também venceu (70,3%), seguido de perto pelo Mask R-CNN (65,1%).

Não há, portanto, um vencedor único e absoluto entre os três métodos — a escolha do “melhor modelo” depende do contexto de aplicação e da métrica que se deseja priorizar. Esse é, a meu ver, o resultado mais interessante e maduro do trabalho: não uma simples afirmação de que “o modelo X é melhor”, mas uma análise comparativa que reconhece os diferentes trade-offs entre os métodos avaliados.

14. Controle de Versão com GitHub

Todo o código do projeto foi versionado e publicado em um repositório público no GitHub (github.com/CaueGhost/cardamage-sar), incluindo um arquivo README.md com a documentação completa do projeto (objetivo, estrutura de pastas, instruções de instalação e execução, e a tabela de resultados), um arquivo requirements.txt com as dependências necessárias, e um arquivo .gitignore configurado para excluir do repositório os arquivos pesados e não essenciais ao código-fonte: o dataset de imagens (por restrição de licença de uso), os modelos treinados em formato .pt (por seu tamanho) e o ambiente virtual .venv. O repositório foi atualizado em múltiplas ocasiões ao longo do processo de correção, acompanhando cada uma das mudanças estruturais descritas neste relatório.

15. Análise Crítica Final e Trabalhos Futuros

Olhando o processo como um todo, identifico três categorias de aprendizado que extrapolam o conteúdo técnico específico de deep learning.

Sobre reprodutibilidade

A variação dos resultados entre execuções diferentes do mesmo script — algumas devido à aleatoriedade inerente ao treinamento, outras devido a bugs introduzidos por edições manuais não controladas — reforçou a importância de versionar e isolar cuidadosamente qualquer alteração de código, além de documentar explicitamente as condições de cada execução (hardware, hiperparâmetros, limiares de confiança).

Sobre comunicação técnica

A primeira apresentação não foi aprovada não por falhas no conteúdo técnico central, mas pela ausência de material visual de apoio e pela demonstração incompleta das funcionalidades do programa. Isso evidencia que a qualidade de um projeto técnico não se mede apenas pela correção do código, mas também pela capacidade de demonstrá-lo de forma clara e completa a quem avalia.

Sobre as limitações conhecidas do trabalho

Registro aqui, de forma transparente, três limitações conhecidas: o dataset de treinamento utilizado é pequeno (59 imagens), o que limita a capacidade de generalização de todos os três modelos; os limiares de confiança utilizados na avaliação não são idênticos entre os modelos (0,1

para o YOLOv8 e 0,5 para os demais), o que favorece artificialmente o Recall do primeiro; e a métrica de IoU, mais rigorosa do que a contagem simples de detecções, não foi implementada.

Trabalhos futuros

- Padronizar o limiar de confiança entre os três modelos, para uma comparação estritamente equivalente.
- Treinar os modelos com a base de dados CarDD, mais robusta, caso uma GPU esteja disponível.
- Incorporar uma análise complementar com o algoritmo K-Means, agrupando os danos por tamanho, forma ou características visuais, como uma camada adicional de análise não supervisionada sobre os resultados da detecção supervisionada.

16. Referências Bibliográficas

- IOFFE, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *arXiv preprint arXiv:1502.03167*.
- JOHNSON, J. (2019). *CS231n: Convolutional Neural Networks for Visual Recognition* [Lecture 7]. Stanford University / Michigan Online.
- LECUN, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.